Content

- Today lecture: Sentiment Analysis Classification Task
  - Problem Definition
  - Features extraction
    - Bag of Word
    - TF-IDF
    - LSA
  - Data Set
  - Perceptron Model training
  - Logistic Regression Model training
  - K-Nearest Neighbors (KNN)
  - Comparisons
  - Assignment # 2

Sentiment Analysis Problem Definition

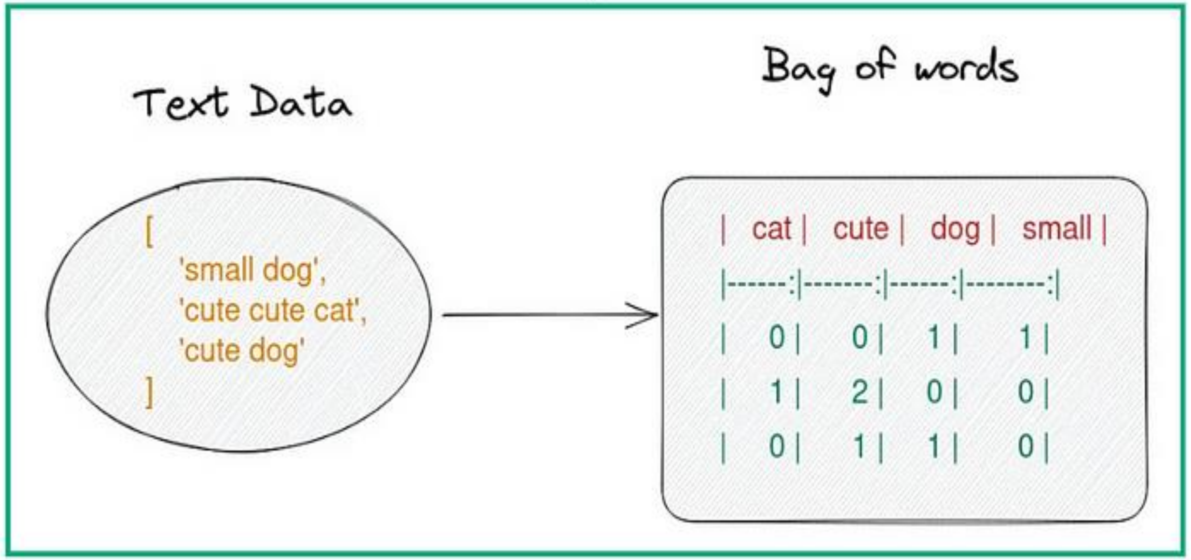
- A basic task in sentiment analysis is to classify the polarity of a given text at the document, sentence, or feature/aspect level
  - Whether the expressed opinion is positive, negative, or neutral.
- Advanced, “beyond polarity” sentiment classification looks, for instance, at emotional states such as “angry”, “sad”, and “happy”.

The Importance of Sentiment Analysis

- Business Insights
  - Sentiment analytics helps in understanding customer sentiments, managing brand reputation, and making informed business decisions.
- Stock Market Predictions
  - used in the financial sector to predict stock market trends.
  - analyzes news articles, social media posts, and financial reports to assess market sentiment, helping investors make more informed decisions.
- Social Media Monitoring
  - essential for monitoring brand mentions, tracking trends, and managing online reputation.
- Political Analysis
  - used to assess public opinion, especially during election campaigns

Bag of Word Features Extraction

	Term 1	Term 2	Term 3	...	Term M
Tweet 1	0	1	1	0	0
Tweet 2	0	1	0	0	0
Tweet 3	0	0	0	3	0
...	0	0	0	1	1
Tweet N	0	0	0	1	0



Bag-of-Words Word Cloud

Positive Tweets Word Cloud

Negative Tweets Word Cloud

Pros and cons of Bag-of-Words

- The bag-of-words model is very simple to understand and implement and offers a lot of flexibility for customization on your specific text data.
- It has been used with great success on prediction problems like language modeling and documentation classification.
- Some bag-of-words limitation:
  - Vocabulary:
    - The vocabulary requires careful design, most specifically in order to manage the size, which impacts the sparsity of the document representations.
  - Sparsity:
    - Sparse representations are harder to model both for computational reasons (space and time complexity).
  - Meaning:
    - Discarding word order ignores the context, and in turn meaning of words in the document (semantics).

TF-IDF Features Extraction

TF-IDF (Term Frequency-Inverse Document Frequency) is a popular feature extraction method used in text mining and information retrieval, including sentiment analysis on Twitter datasets. Here's a brief overview of how it works and its application:

How TF-IDF Works

1. Term Frequency (TF): Measures how frequently a term appears in a document. The assumption is that the more a term appears, the more important it is.

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. Inverse Document Frequency (IDF): Measures how important a term is. While TF considers all terms equally important, IDF reduces the weight of terms that appear very frequently across all documents and increases the weight of terms that appear rarely.

$$IDF(t) = \log\left(\frac{\text{Total number of documents}}{\text{Number of documents with term } t}\right)$$

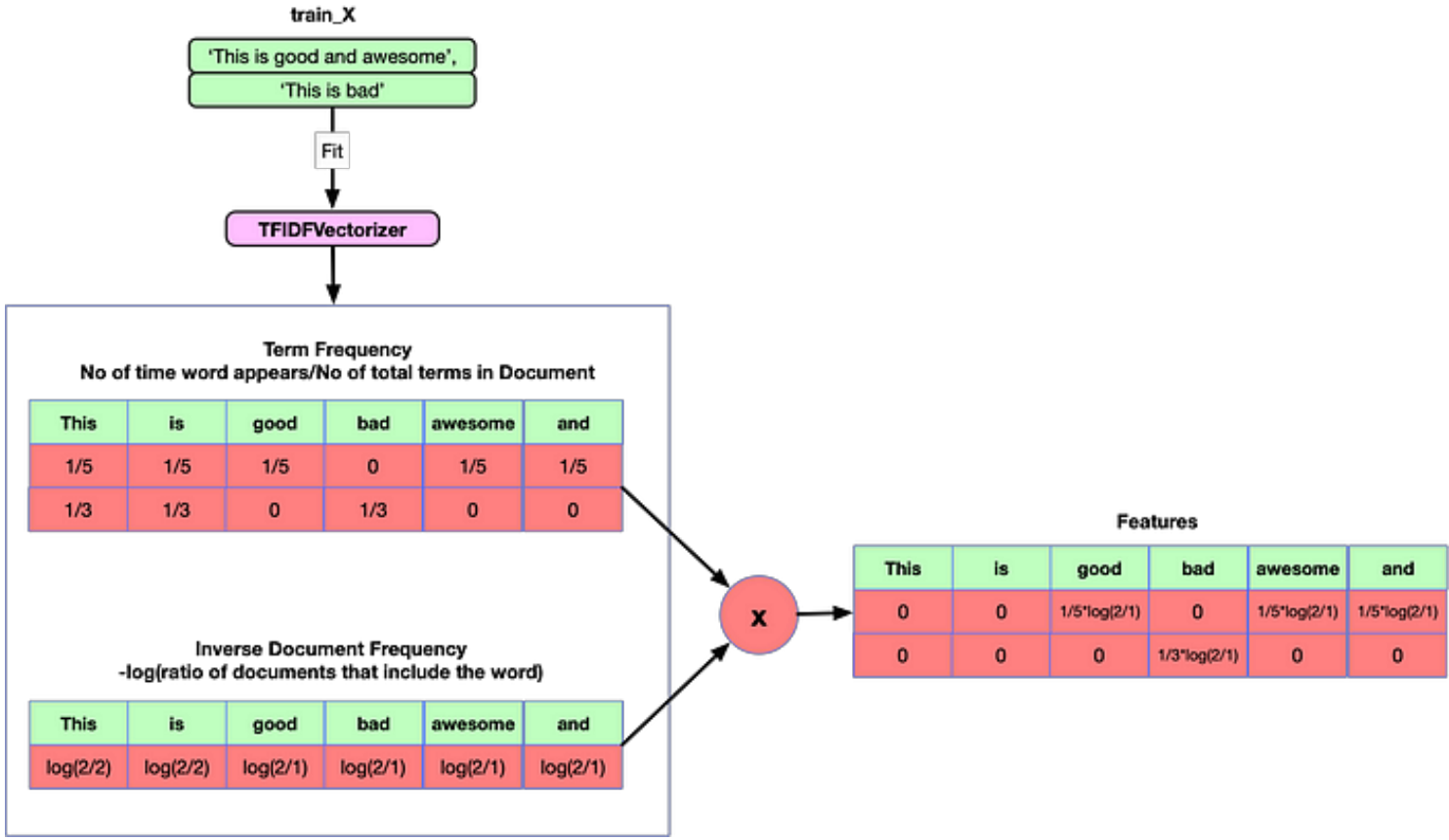
3. TF-IDF Score: The product of TF and IDF, giving a score that balances the term's frequency in a specific document with how common the term is across all documents.

$$\text{TF-IDF}(t,d) = TF(t,d) \times IDF(t)$$

TF-IDF Matrix Construction

A = “The car is driven on the road”;
B = “The truck is driven on the highway”

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	1/7	1/7	log(2/2) = 0	0	0
Car	1/7	0	log(2/1) = 0.3	0.043	0
Truck	0	1/7	log(2/1) = 0.3	0	0.043
Is	1/7	1/7	log(2/2) = 0	0	0
Driven	1/7	1/7	log(2/2) = 0	0	0
On	1/7	1/7	log(2/2) = 0	0	0
The	1/7	1/7	log(2/2) = 0	0	0
Road	1/7	0	log(2/1) = 0.3	0.043	0
Highway	0	1/7	log(2/1) = 0.3	0	0.043



Pros and cons of TF-IDF

Advantages of TF-IDF

- Simplicity: TF-IDF is easy to understand and implement. It provides a straightforward way to quantify the importance of a term in a document relative to a collection of documents.
- Relevance: By weighing terms based on their frequency in a document and their rarity across a corpus, TF-IDF helps identify the most relevant terms for a given document.
- Effectiveness in Information Retrieval: It is effective in search engines and document retrieval systems, as it helps rank documents based on their relevance to a search query.

Disadvantages of TF-IDF

- Context Ignorance: TF-IDF does not consider the context in which terms appear. Words with multiple meanings (polysemy) or synonyms may be treated as separate entities, leading to potential misinterpretation.
- Document Length Sensitivity: Longer documents may have higher term frequencies, which can skew the importance of terms.
- Lack of Semantic Understanding: TF-IDF does not capture the semantic meaning of words or phrases, which can limit its effectiveness in understanding the content of documents.
- Sparse Representation: The resulting vectors can be very sparse, especially in large corpora, which can make them less efficient for certain machine learning algorithms.

LSA Features Extraction

- Latent Semantic Analysis (LSA) is a popular, dimensionality-reduction techniques
- It uses the method as Singular Value Decomposition to decompose large matrix into three small components matrices.
- LSA can be used to reformulate tf-idf matrix in terms of r latent features, where r is less than m
  - m: the number of terms in the data used in tf-idf matrix (n x m) with n number of documents
- r could be reduced to a lower value by removing less important features which has small values in S matrix

```
Tfidf_m = TfidfVectorizer(max_features=5000)
tfidf = Tfidf_m.fit_transform(doc)
Lsa_m = TruncatedSVD(n_components=100)
lsa = lsa_m.fit_transform(tfidf)
```

A = U x S x Vt

	d1	d2	d3	d4
a	6	7	1	0
b	8	6	0	1
c	6	9	8	5
d	0	1	8	8
e	2	0	9	7
f	2	0	7	7

	f1	f2	f3	f4
a	0.24	-0.51	0.08	0.06
b	0.25	-0.54	-0.64	-0.23
c	0.58	-0.28	0.57	0.13
d	0.42	0.37	0.16	-0.68
e	0.44	0.34	-0.24	0.66
f	0.39	0.29	-0.40	-0.09

	f1	f2	f3	f4
f1	23.1	0	0	0
f2	0	14.3	0	0
f3	0	0	3.5	0
f4	0	0	0	1.5

	d1	d2	d3	d4
f1	0.37	0.38	0.65	0.53
f2	-0.55	-0.63	0.37	0.38
f3	-0.69	0.59	0.27	-0.21
f4	0.26	-0.29	0.59	-0.69

Pros and cons of LSA

Advantages

- Dimensionality Reduction: LSA reduces the dimensionality of the term-document matrix, which helps in simplifying data and improving computational efficiency.
- Captures Synonymy: By analyzing the context in which words appear, LSA can identify synonyms and related terms, enhancing the understanding of text.
- Handles Polysemy: LSA can help disambiguate words with multiple meanings by considering the overall context in which they are used.
- Semantic Understanding: It provides a way to capture the underlying semantic structure of the text, allowing for better information retrieval and document similarity assessments.
- Robustness to Noise: LSA can be more robust to noise in text data, as it focuses on patterns and relationships rather than exact word matches.

Disadvantages

- Interpretability: The resulting latent factors from LSA can be difficult to interpret, making it challenging to understand the specific meanings of the dimensions.
- Data Sparsity: LSA relies on large corpora to be effective. In smaller datasets, the latent structure may not be as clear or significant.
- Computational Complexity: While LSA reduces dimensionality, the initial computation of the singular value decomposition (SVD) can be computationally intensive, especially for large datasets.
- Assumption of Linearity: LSA assumes linear relationships between terms and documents, which may not always hold true in natural language.
- Contextual Limitations: LSA does not account for word order or the syntactic structure of sentences, potentially losing important contextual information.

Data Set

- Public data set on tweets in 2015
- Provider: Natural Language Toolkit (NLTK)
- Data Set Size : 10000
- Train set 8000
- Test set 2000
- 5000 negative sentiment
- 5000 positive sentiment

	text	sentiment
0	RT @NancyLeeGrahn: How did everyone feel about...	Neutral
1	RT @ScottWalker: Didn't catch the full #GOPdeb...	Positive
2	RT @TJMShow: No mention of Tamir Rice and the ...	Neutral
3	RT @RobGeorge: That Carly Fiorina is trending ...	Positive
4	RT @DanScavino: #GOPDebate w/ @realDonaldTrump...	Positive

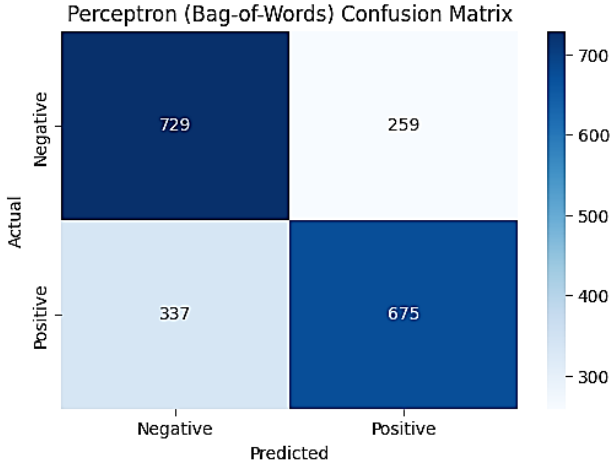
Text Preprocessing

```
# here i am cleaning text column
def cleantxt(text):
    text= re.sub(r'@[A-Za-z0-9]+', '',text)# removed @mentions
    text= re.sub(r'#', '',text)# removed # symbol
    text = re.sub(r'RT[\s]+', '',text)# rmoved RT
    text = re.sub(r'https?:\/\/\s+', '',text)# removed the hyperlink
    text = re.sub(r':+', '',text)# removed : symbol
    text = re.sub(r'--+', '',text)# removed : symbol
    text = re.sub(r'http', '',text)
    return text
data["text"] = data["text"].apply(cleantxt)
```

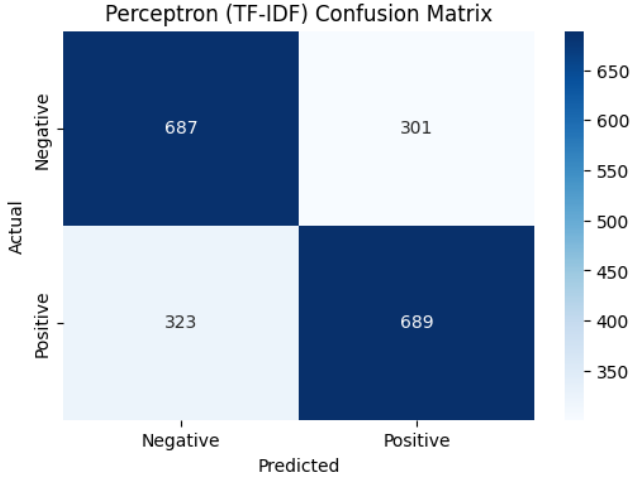
	text	sentiment
0	How did everyone feel about the Climate Chang...	Neutral
1	Didn't catch the full GOPdebate last night. H...	Positive
2	No mention of Tamir Rice and the GOPDebate wa...	Neutral
3	That Carly Fiorina is trending hours after H...	Positive
4	GOPDebate w/ delivered the highest ratings i...	Positive

Perceptron Model

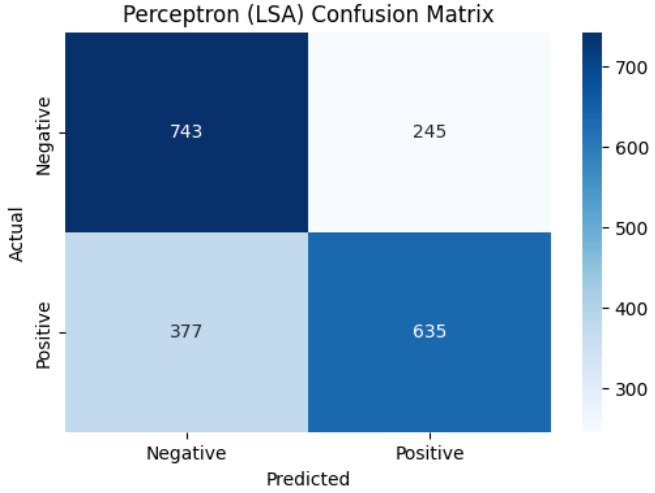
Bag-of-Words	precision	recall	f1-score	support
0.0	0.68	0.74	0.71	988
1.0	0.72	0.67	0.69	1012
accuracy			0.70	2000
macro avg	0.70	0.70	0.70	2000
weighted avg	0.70	0.70	0.70	2000



TF-IDF	precision	recall	f1-score	support
0.0	0.68	0.70	0.69	988
1.0	0.70	0.68	0.69	1012
accuracy			0.69	2000
macro avg	0.69	0.69	0.69	2000
weighted avg	0.69	0.69	0.69	2000

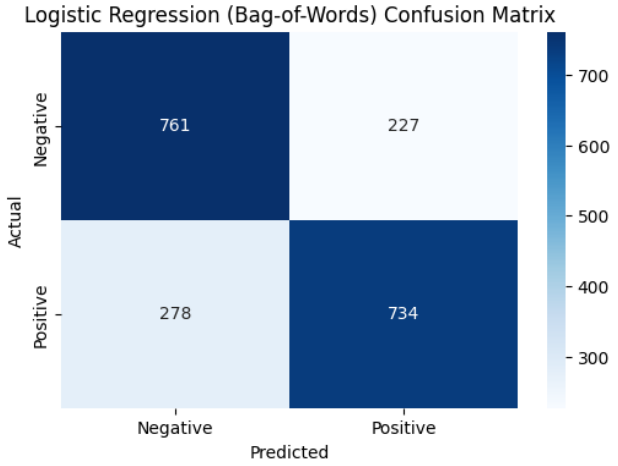


LSA	precision	recall	f1-score	support
0.0	0.66	0.75	0.70	988
1.0	0.72	0.63	0.67	1012
accuracy			0.69	2000
macro avg	0.69	0.69	0.69	2000
weighted avg	0.69	0.69	0.69	2000

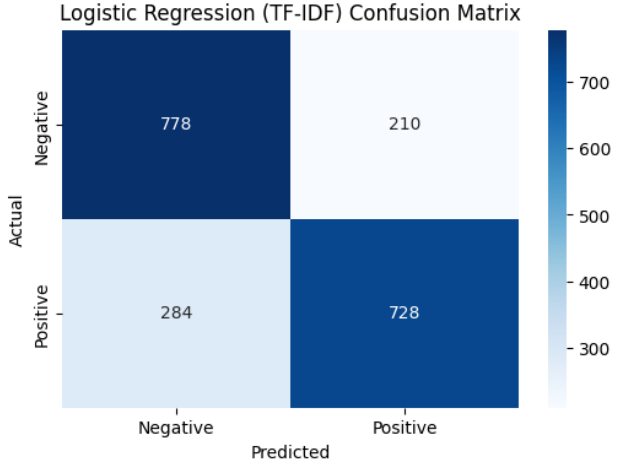


Logistic Regression Model

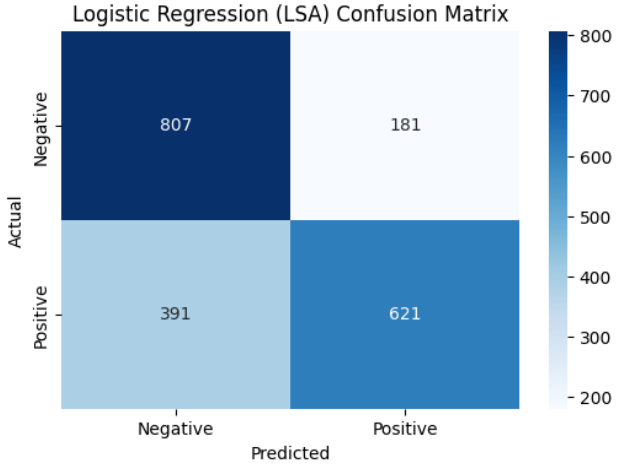
Bag-of-Words	precision	recall	f1-score	support
0.0	0.73	0.77	0.75	988
1.0	0.76	0.73	0.74	1012
accuracy			0.75	2000
macro avg	0.75	0.75	0.75	2000
weighted avg	0.75	0.75	0.75	2000



TF-IDF	precision	recall	f1-score	support
0.0	0.73	0.79	0.76	988
1.0	0.78	0.72	0.75	1012
accuracy			0.75	2000
macro avg	0.75	0.75	0.75	2000
weighted avg	0.75	0.75	0.75	2000

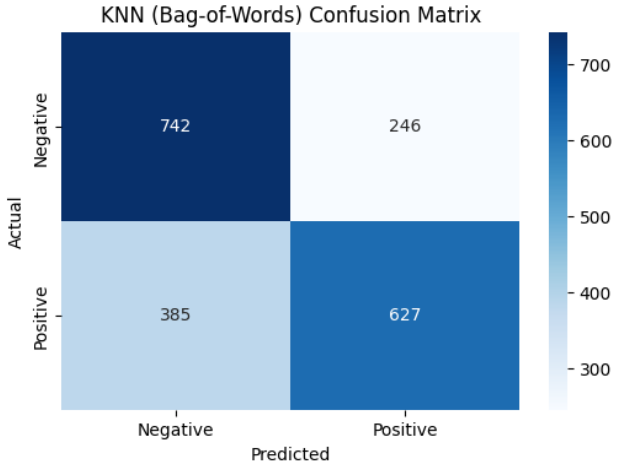


LSA	precision	recall	f1-score	support
0.0	0.67	0.82	0.74	988
1.0	0.77	0.61	0.68	1012
accuracy			0.71	2000
macro avg	0.72	0.72	0.71	2000
weighted avg	0.72	0.71	0.71	2000

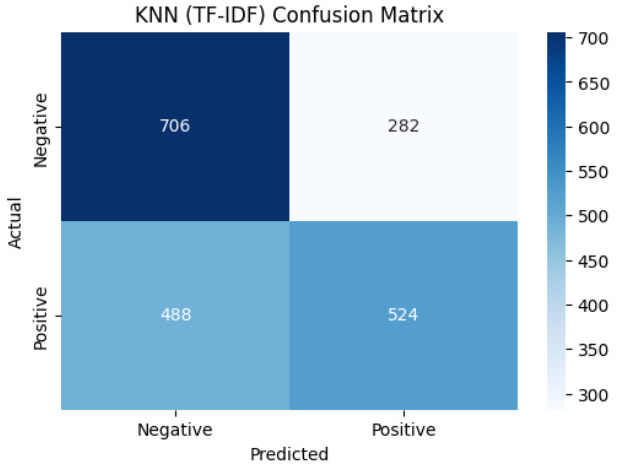


K-Nearest Neighbors Model

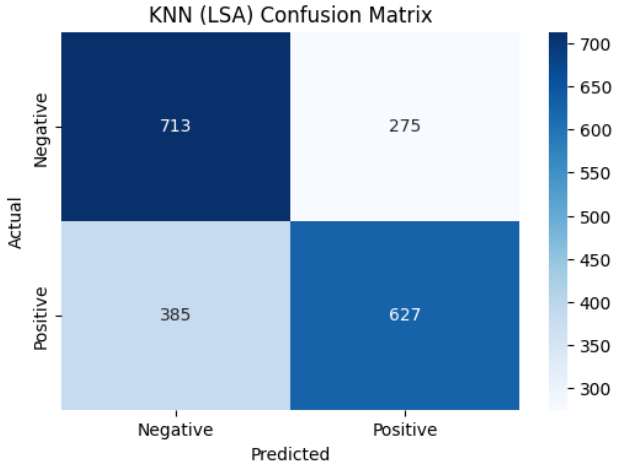
Bag-of-Words	precision	recall	f1-score	support
0.0	0.66	0.75	0.70	988
1.0	0.72	0.62	0.67	1012
accuracy			0.68	2000
macro avg	0.69	0.69	0.68	2000
weighted avg	0.69	0.68	0.68	2000



TF-IDF	precision	recall	f1-score	support
0.0	0.59	0.71	0.65	988
1.0	0.65	0.52	0.58	1012
accuracy			0.61	2000
macro avg	0.62	0.62	0.61	2000
weighted avg	0.62	0.61	0.61	2000



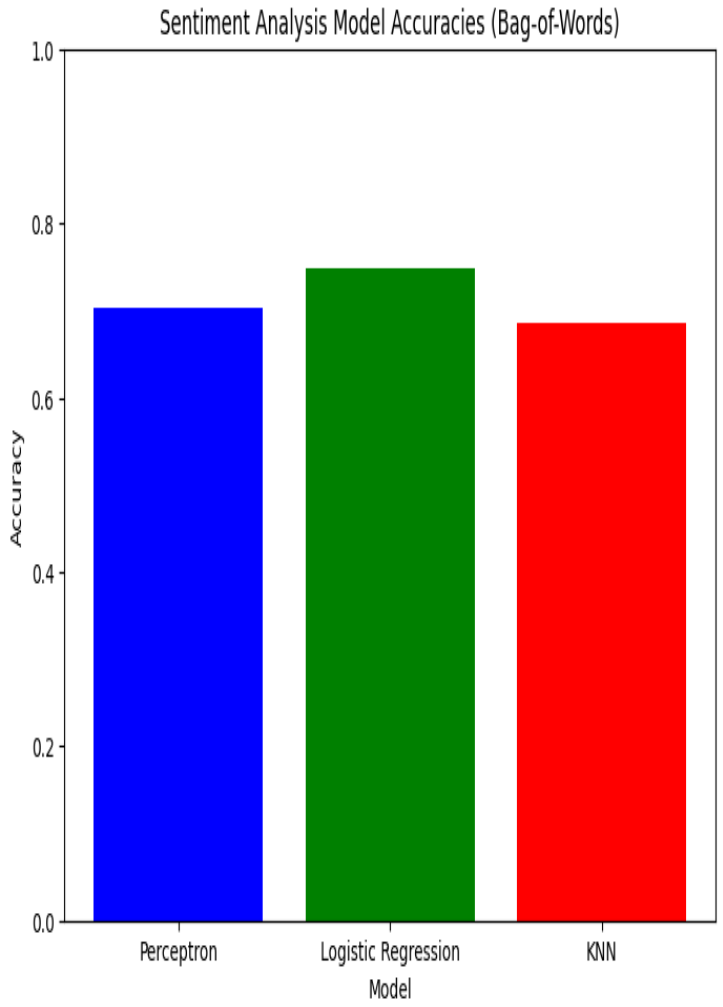
LSA	precision	recall	f1-score	support
0.0	0.65	0.72	0.68	988
1.0	0.70	0.62	0.66	1012
accuracy			0.67	2000
macro avg	0.67	0.67	0.67	2000
weighted avg	0.67	0.67	0.67	2000

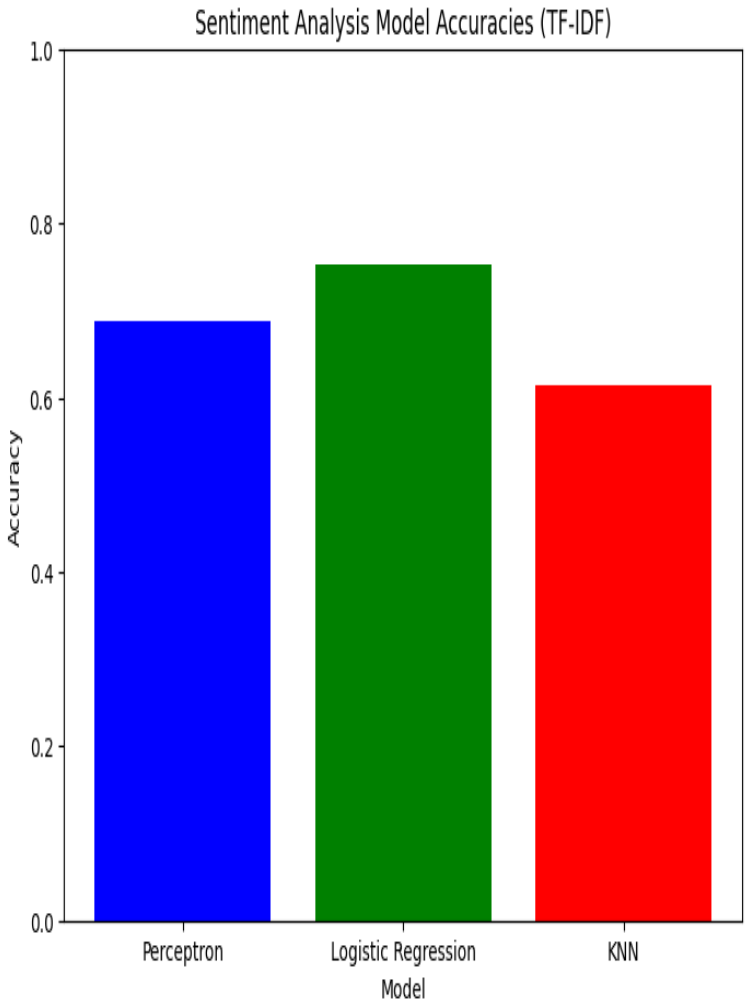


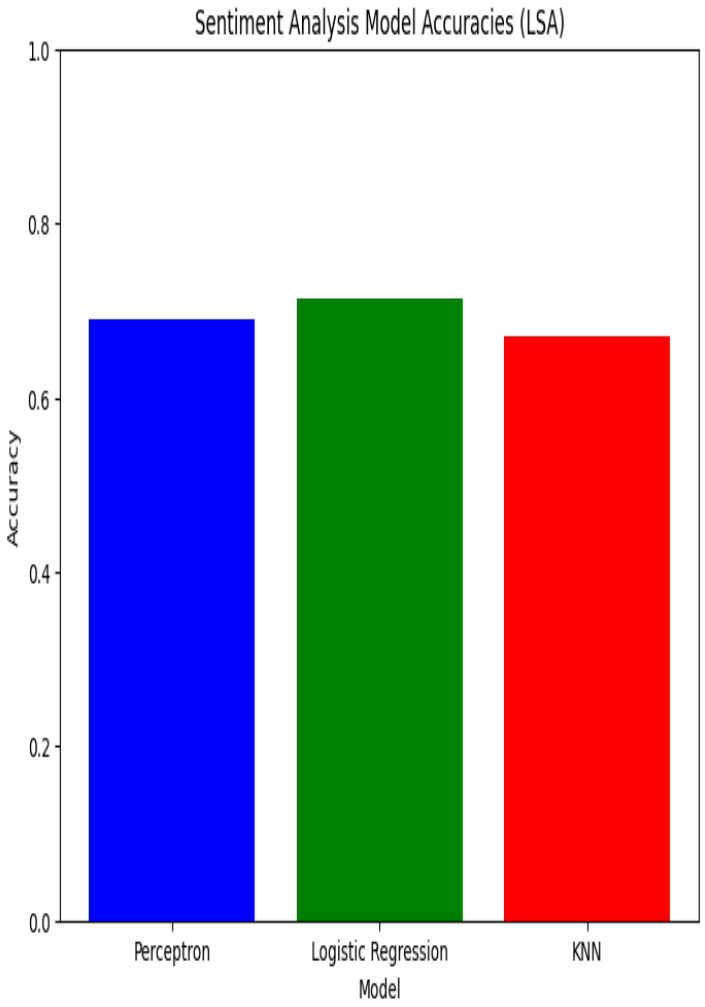
Comparisons

Matrix	Feature Extraction	Perceptron	Logistic Regression	KNN
Accuracy	Bag of Word	0.70	**0.75**	0.68
	TF-IDF	0.69	**0.75**	0.61
	LSA	0.69	**0.71**	0.67
Precision	Bag of Word	0.70	**0.75**	0.69
	TF-IDF	0.69	**0.75**	0.62
	LSA	0.69	**0.72**	0.67
Recall	Bag of Word	0.70	**0.75**	0.68
	TF-IDF	0.69	**0.75**	0.61
	LSA	0.69	**0.71**	0.67

Comparisons







Assignment # 2

- Using the same data set, extend the work done with Hashing Vectorizer HV method with vector size of 1024

Matrix	Feature Extraction	Perceptron	Logistic Regression	KNN
Accuracy	Bag of Word	0.70	**0.75**	0.68
	TF-IDF	0.69	**0.75**	0.61
	LSA	0.69	**0.71**	0.67
	Hashing Vectorizer (HV)			
Precision	Bag of Word	0.70	**0.75**	0.69
	TF-IDF	0.69	**0.75**	0.62
	LSA	0.69	**0.72**	0.67
	Hashing Vectorizer (HV)			
Recall	Bag of Word	0.70	**0.75**	0.68
	TF-IDF	0.69	**0.75**	0.61
	LSA	0.69	**0.71**	0.67
	Hashing Vectorizer (HV)			

THANK YOU

Q & A